Fusion Engineering and Design 127 (2018) 202–206

Contents lists available at ScienceDirect

Fusion Engineering and Design

journal homepage: www.elsevier.com/locate/fusengdes

Development of a real-time data archive system for a KSTAR real-time network

Giil Kwon*, Woongryol Lee, Taegu Lee, Jaesic Hong

Control Team, National Fusion Research Institute, Daejeon, South Korea

ARTICLE INFO

Keywords:
Real-time data translation
Network
Reflective memory (RFM)
MDSplus

ABSTRACT

This paper presents the Korea Superconducting Tokamak Advanced Research (KSTAR) Real-Time Data Archiving System (RTDAS) for a real-time network. In most fusion experiments, a stable and low latency real-time network is essential for achieving real-time controllability. Inspecting and supervising the operation of a real-time network is essential for ensuring its stability. Archiving the network data makes it possible to analyze the performance and status of a network. In the KSTAR, RTDAS has been developed to monitor and archive real-time network data that is used to control the plasma and send diagnostic data to Plasma Control System (PCS) in real-time to ensure the stable operation of the real-time network. The KSTAR adopts reflective memory (RFM) as a real-time network and this paper presents the algorithm of the real-time data archiving system and the experimental results of this system. The real-time data archiving system operates in two phases. During the experiment, the archiving system stores data from these real-time networks in real-time. After the experiment is finished the system sends this archived data to the MDSplus data server; this system stores all data in accordance with the KSTAR timing system. Therefore, all data is synchronized with the KSTAR timing system. This system is configured with Experimental Physics and Industrial Control System (EPICS) and Asynchronous Driver Support (AsynDriver). This system operates in real-time, was implemented on Messaging Realtime Grid -Real time (MRG-R) kernel 3.10, and uses Multi-Core Utilities (MCoreUtil). The system was tested to validate the performance of this archiving system; the test results show this system's real-time performance.

1. Introduction

Real-time control is essential in nuclear fusion experiments. In particular, in magnet confinement–based fusion systems, PCS connects to the Magnet Power Supply (MPS) in a real-time network to control the plasma parameters. The feedback control used in PCS requires both a fast response time ($< 50\,\mu s$) and stable operation. Achieving this response time requires that PCS and other related devices connect with a low-latency real-time network ($< 50\,\mu s$). The real-time network must be stable to obtain the stability of the feedback system. The stable operation of a real-time network requires regular inspection and supervision. KSTAR uses a RFM network as a real-time network to control MPS because RFM has a low-latency ($< 50\,\mu s$) and provides the deterministic behavior. Each connected node can share up-to-date data in predefined memory areas and each RFM node copies the data written in its shared local memory to all other RFM nodes at a transfer rate up to 170MB/s without CPU overheads. Because of these characteristics, many of tokamak adopt the RFM as real -time network [1–5]. However, an RFM network is vulnerable to data corruption and is difficult to monitor and administer. Because when any RFM node writes the data to RFM shared memory, the RFM node does not know which node wrote the data. Therefore, any RFM node in the network can change shared data without any other node knowing. This causes data corruption and it is often difficult to find which node caused a problem. RFM does not provide network status information; thus, it can be difficult to determine an RFM network's connection status. To prevent theses problem, many of tokamak archive RFM data. QUEST [4] and SST-1 [5] stores RFM data to local file system and share these data from the master node that controls entire system. However, dedicated RFM network archiving system is essential to monitor RFM network. To properly monitor the system, a third-party system must monitor the entire system. If the master node has fault, it is difficult to analyze the fault situation. In KSTAR, Real time monitoring for RFM interface (RTMON) was developed at 2012 [6]. RTMON convert RFM data to EPICS [7] PV. and archives RFM data to local file system. This system stores the data at low rate(< 1 kHz). However, in order to analyze the plasma phenomenon, a faster data archiving ability (> 1 kHz) is required. This system does not have an interface for sharing data.

* Corresponding author.
E-mail address: giilkwon@nfri.re.kr (G. Kwon).

https://doi.org/10.1016/j.fusengdes.2018.01.019
Received 15 June 2017; Received in revised form 22 December 2017; Accepted 3 January 2018
Available online 09 January 2018
0920-3796/

Therefore, it is hard to share and view this data. KSTAR has developed a real-time data archiving system (RTDAS) to store RFM data. The RTDAS stores RFM data in an MDSplus [8] server. These archived data can be viewed by jScope and python. jScope is a java based tool used to display data stored in mdsplus. In KSTAR, RFM data is stored to local memory during a shot. The RFM data archiving system stores RFM data at 2 kHz in local memory. The server synchronizes with a Time Communication Network (TCN) and runs at 2 kHz. TCN is an ITER Time Communication Network for absolute synchronization that is also synchronized with the KSTAR timing system [9]. Therefore, all archived data are synchronized with the KSTAR timing system. After a shot, the system transfers stored data to the KSTAR MDSplus server through the 1-GbE network.

In the current section, we wrote about an RFM data archiving system. We describe the RFM network in KSTAR in Section 2 and we describe the real-time network data archiving system (RTDAS) in Section 3. We describe performance test environment and the results of the test in Section 4 and Section 5 concludes this paper.

2. RFM network in KSTAR

KSTAR adopted RFM as a real-time network to control MPS and obtain plasma diagnostic information from a diagnostic system in a real-time manner. In the 2017 campaign, 35 RFM nodes were used to control the KSTAR system and PCS and MPS were connected to the RFM network. The fastest control cycle is set to 50 μs. The RFM card always updates the content of the RFM memory and each RFM node copies the data written in shared local memory to all other RFM nodes. RFM read/write throughputs are different; the RFM write throughputs is relatively lower than read throughput. Fig. 1. shows that the KSTAR RFM network has a tree topology; each RFM node is connected to its local RFM hub, and each RFM node and local hub is connected to the KSTAR main RFM hub.

3. Real-time data archive system (RTDAS)

The real-time data archive system (RTDAS) stores RFM data to KSTAR MDSplus server. RTDAS reads data from the RFM shared memory in accordance with the KSTAR timing system and stores data in local memory during a shot. After the shot finishes, the real-time data archive system sends this data to the MDSplus server. An analysis of the archived data permits measurement of the RFM network performance and detection of faults.

3.1. Real-time parallel data communication and processing software framework (RT-ParaPro)

RTDAS was developed using real time parallel data communication and a processing software framework (RT-ParaPro) [10]. This software framework provides real time data communication using a real-time network (such as SDN or RFM) and parallel data processing in a real-time manner. As shown in Fig. 2, RT-ParaPro provides three networks interfaces (SDN, RFM, and EPICS). Using the EPICS channel access interface means that system-required parameters can be set and the system status can be monitored from a remote location.

Synchronous Data Network (SDN) and RFM can be used as a real-time network [9]. RT-ParaPro uses multi thread pairs to process multiple data in parallel. And each thread pair implements production and consumption patterns; one pair consists of three elements: a producer thread, circular buffer, and consumer thread. The producer thread produces data by receiving data from network and pushing the data to the circular buffer. The consumer thread get data from the circular buffer and sent through a network interface. Since the two threads operate asynchronously, this system can efficiently process data from networks with different data rates. With this framework, RTDAS can efficiently store data coming from the network at a higher data transfer rate than writing data to local storage.

3.2. Architecture of a real-time data archive system

Fig. 3 describes the architecture of a RTDAS. This system was written on top of MRG-R [11] kernel–based 64-bit Scientific Linux 6.5 to provide a real-time performance RTDAS that consists of multiple

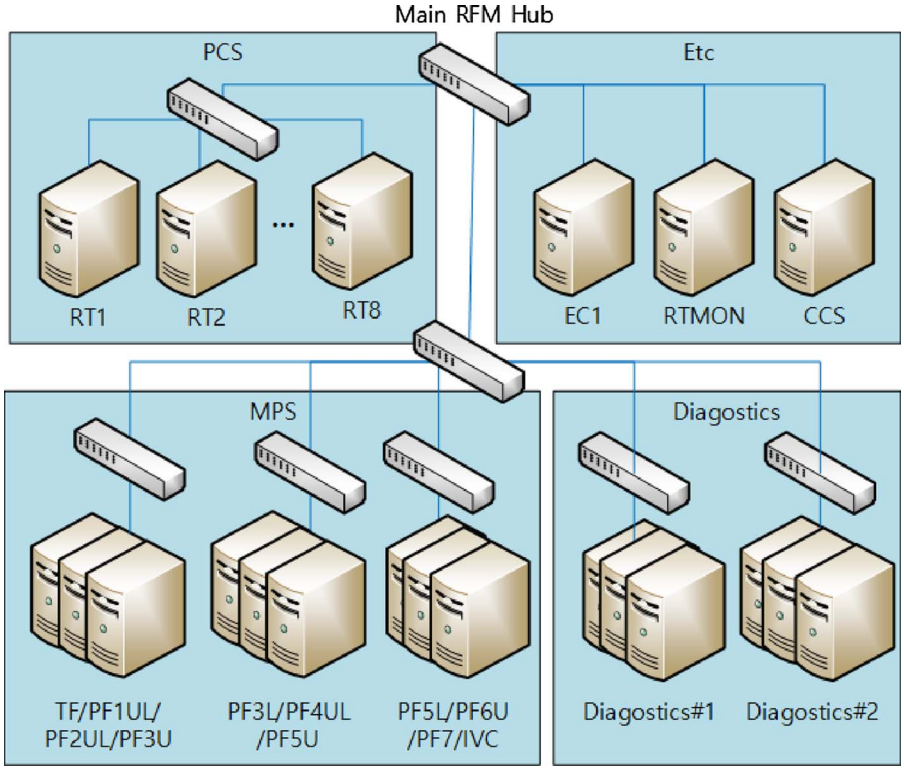

Fig. 1. KSTAR RFM network configuration.

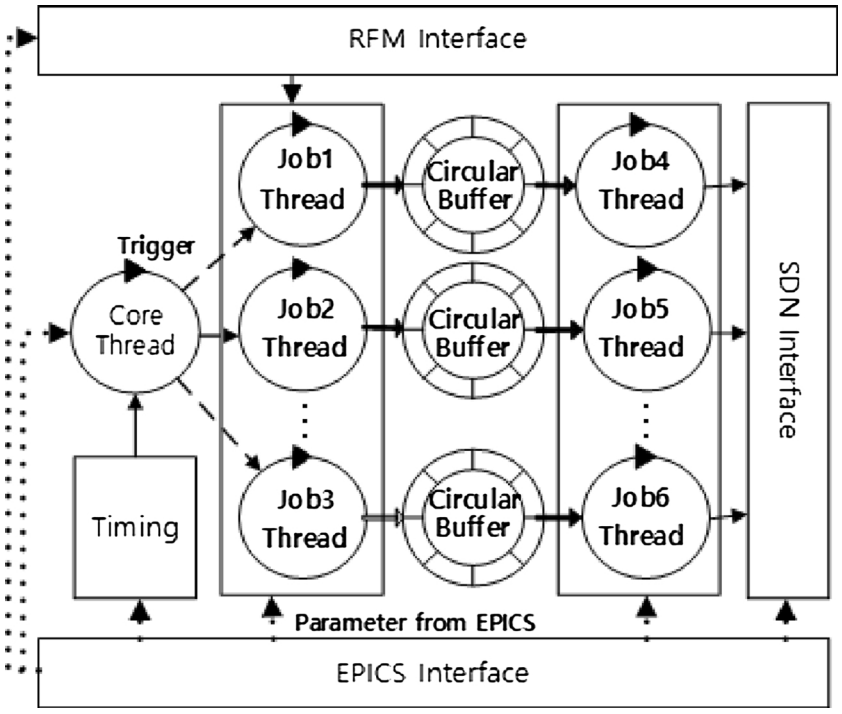

Fig. 2. Real-time parallel data communication and processing software framework (RT-ParaPro).

threads to process data in parallel. The system uses Experimental Physics and Industrial Control System (EPICS) to send and receive system parameter information remotely. Each CPU core is isolated and assigned to predefined thread. McoreUtil [12] is used to assign and manage thread priority and schedule policies. AsynDriver [13] allows accessing EPICS Process Variables (PV) in a non-blocking mode that does not degrade the real-time performance of IOC applications. The server is connected to TCN, which is an Ethernet network that is compliant with IEEE 1588–2008.

TCN is also synchronized with the KSTAR timing system. Therefore, the archived data are synchronized with the KSTAR time system.

3.3. Application structure

Fig. 4 describes the structure of RTDAS. Each thread receives a trigger from the core thread and the core thread generates a trigger at a frequency of 2 kHz by receiving a clock from the TCN. Therefore, each data point is synchronized with the KSTAR timing system. In RTDAS, each RFM memory address is mapped to predefined MDSplus tag. During a shot, each production thread stores data from the predefined RFM address to the local memory. After the shot finishes, RTDAS sends this data to the MDSplus server with predefined MDSplus tag.

4. Performance test

We performed a core thread period consistency test to evaluate the real-time performance of RTDAS.

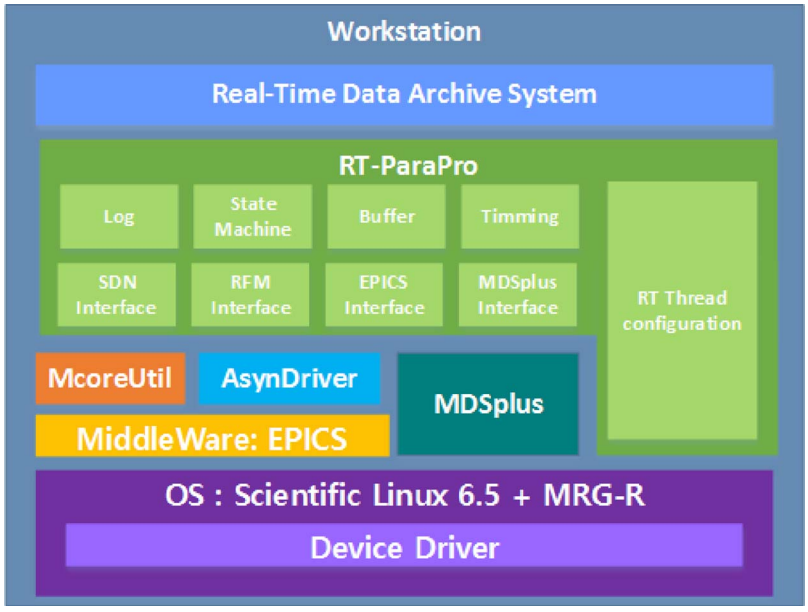

Fig. 3. Architecture of the real-time data archive system.

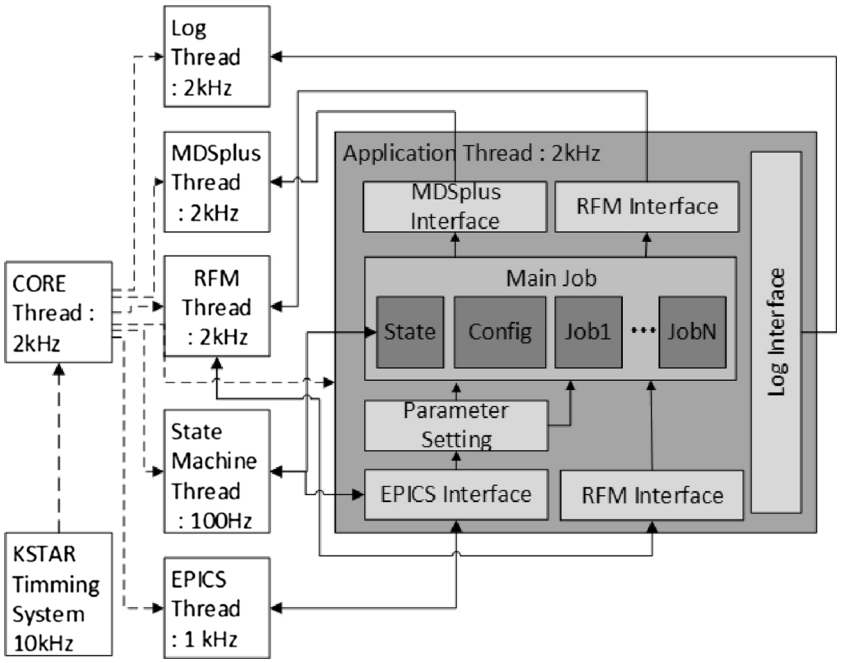

Fig. 4. Real-time data archive system structure.

4.1. Test environment and method

Thread period consistency can be considered a key measure of a deterministic real-time system. The loop period consistency can be measured by calculating the time difference between the target loop start time and the measured start time of a loop. We tested RTDAS with another node that simulate the PCS voltage commands data. As Fig. 5 shows us, RTDAS is connected to an RFM network to update the shared data and connected to 1 G Ethernet to send archived data to the MDSplus server. And RFM writer node also connected to RFM. RFM writer node simulate PCS command data. Among the PCS command data, we simulate and archive PCS voltage command data that sent to poloidal field (PF) coil system. Because, these data are important. And The data transmitted from the PCS has a similar form. The RFM writer node get PCS voltage command data from MDSplus and write these data to RFM at 5 kHz. (Every 200 μs, the PCS write MPS command data to RFM) RTDAS and RFM Writer nodes are synchronized with TCN. The RTDAS server consists of two sockets for Intel Xeon E5 2.4 GHz CPU which has 6 cores. Among 12 CPU cores, we isolated 10 cores. These cores are used to run real time application. RFM Writer node has Intel Core i7 3.6 GHz CPU that has 4 cores. RFM Writer node run Scientific Linux 7 with RT(RealTime). We also isolate the cores for real time applications. In this test, RTDAS archive the data to MPS and In-vessel control coil Power Supply (IPS) from RFM writer. This data size is 256 bytes. Among that data, RFM writer node only update data to MPS (the command data size is 144 bytes). This is due to limited writing speed of our RFM system. As Fig. 4 show us, core thread and the other threads of RTDAS ran at 2 kHz (with a 500 μs period). These threads are synchronized with KSTAR timing system. The experiment was repeated 100,000 times to ensure the accuracy of the test results.

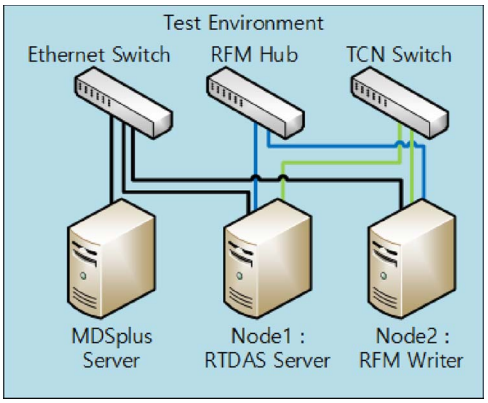

Fig. 5. RTDAS test environment.

4.2. Test result

Fig. 6 represents the distribution of the period of this system; We estimates the distribution of period. We fit the data to Gaussian distribution. The mean was 500 μs, the standard deviation was 0.51 μs, and the jitter was 6.84 μs. The period of this system is between 496.58 μs and 503.42 μs in 100,000 times run. Considering that the scheduling jitter of MRG-R is 2–4 μs, this system has sufficiently low jitter [11].

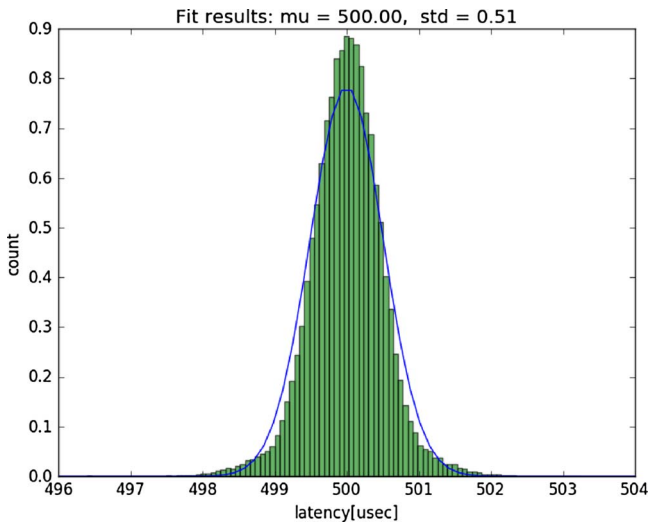

Fig. 6. The RTDAS core thread period distribution.

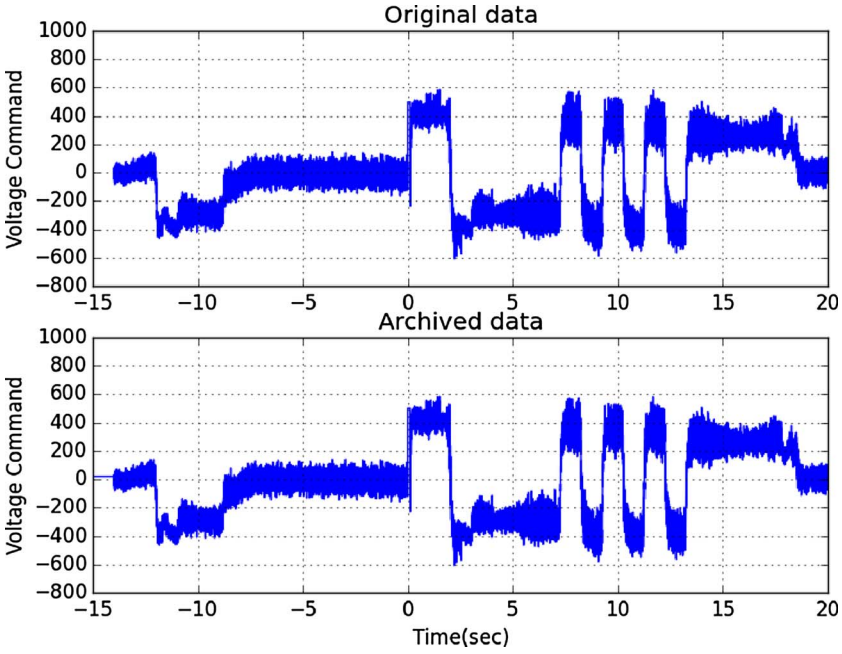

Fig. 7. Archived RFM data at KSATR 17650 shot; Upper figure show us the original data from simulated PCS command and lower figure represents the archived data by proposed system.

4.3. Experiment result

The real-time performance of this system was evaluated by archiving the simulated PF voltage command data from RFM writer node. Fig. 7 shows the result stored in our system. Upper figure represents original data from MDSplus. And lower figure shows us the archived data from RTDAS. RFM writer node simulates PCS voltage command data at KSTAR 17650 shot. These two signals have completely same shape. RTDAS effectively stores RFM data to an KSTAR MDSplus server at 2 kHz.

5. Conclusion

This paper introduces a Real-Time Data Archive System (RTDAS) that stores RFM data to MDSplus server. This system stores RFM data at 2 kHz to system memory during shot to send archived data to MDSplus server after the shot. RTDAS was developed using a real time parallel data communication and processing software framework (RT-ParaPro). This system has deterministic behavior (jitter was 6.84 μs at 2 kHz). RTDAS effectively stores RFM data to an MDSplus server at 2 kHz. Currently, the data can be accessed after archived data transferred to MDSplus server. We will implement the service that can access data while simultaneously archiving data.

Acknowledgments

This work was performed within the cooperation defined by the Memorandum of Understanding (MoU) between the ITER International Organization (IO) and NFRI, and was partially supported by the Korean Ministry of Science ICT & Future Planning under the KSTAR project.

References

[1] H.B. Le, et al., Distributed digital real-time control system for TCV tokamak, Fusion Eng. Des. 89 (3) (2014) 155–164.
[2] Q.P. Yuan, et al., New achievements in the EAST plasma control system, Fusion Eng. Des. 85 (3) (2010) 474–477.
[3] Sang-hee Hahn, et al., Progress and plan of KSTAR plasma control system upgrade, Fusion Eng. Des. 112 (2016) 687–691.
[4] Makoto Hasegawa, et al., Development of a high-performance control system by decentralization with reflective memory on QUEST, Fusion Eng. Des. 96 (2015) 629–632.
[5] Kirit Patel, et al., Design and architecture of SST-1 basic plasma control system, Fusion Eng. Des. 112 (2016) 703–709.
[6] Woongryol Lee, et al., Design and implementation of a standard framework for KSTAR control system, Fusion Eng. Des. 89 (5) (2014) 608–613.
[7] Leo R. Dalesio, A.J. Kozubal, M.R. Kraimer, EPICS Architecture, Los Alamos National Lab., NM (United States), 1991.
[8] Thomas W. Fredian, Joshua A. Stillerman, MDSplus. Current developments and future directions, Fusion Eng. Des. 60 (3) (2002) 229–233.
[9] Giil Kwon, et al., Development of real-time network translator between ITER synchronous data bus network and reflective memory, Fusion Eng. Des. 123 (2017) 955–959.
[10] Giil Kwon, et al., A real-time parallel data communication and processing software framework, KSTAR Conference (2017).
[11] Red Hat Enterprise MRG – Realtime Whitepaper, 2007.
[12] Ralph Lange II, BESSY. Optimizing EPICS for Multi-Core Architectures, (2018).
[13] Martin Kraimer, Mark Rivers, Eric Norum, EPICS: Asynchronous driver support, ICALEPCS (2009) 074–075.